



Week 5 - Selection and Iteration

What is in this lesson?

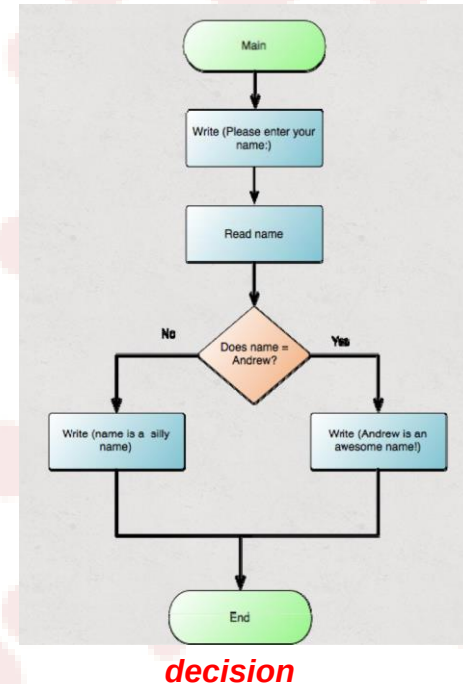
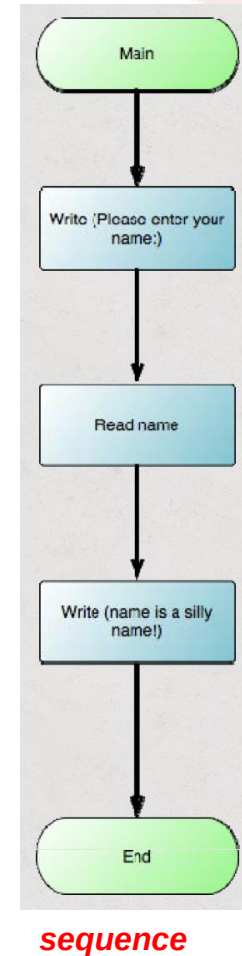
1. The Decision Structure
 - **The Relational Operators**
 - **The if Statement**
 - **The if-else Statement**
 - **Logical Operators**
 - **The Match Case in Python version 3.0**
2. Introduction to Repetition Structures
 - **The while Loop**
 - **The for Loop:**
3. Processing List with Menu

Decision Structures and Boolean Logic

1. The Decision Structure

A **control structure** is a logical design that controls the order in which a set of statements execute.

- A **sequence structure** is a set of statements that execute in the order that they appear.
- A **decision** (or **selection**) **structure** is a control structure that can execute a set of statements and perform a specific action only if a certain condition exists.



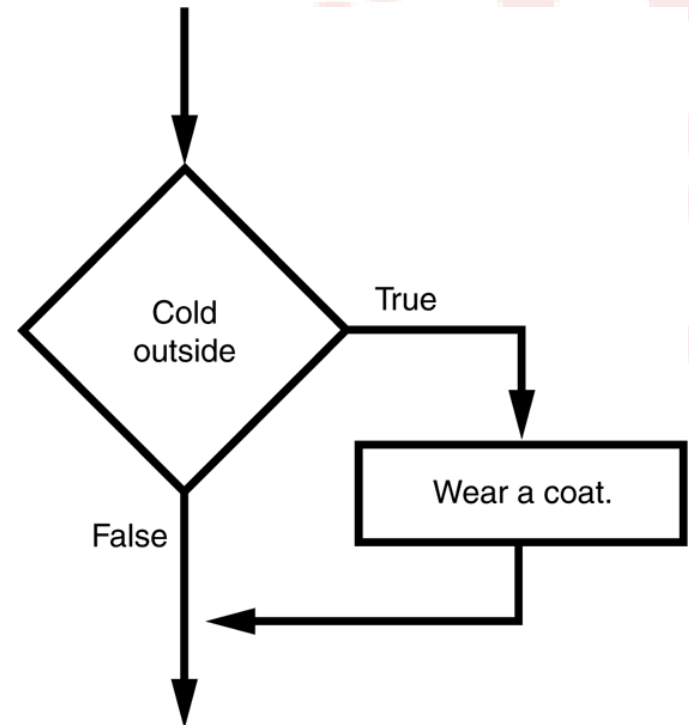
The Decision Structure

Decision (selection) structure:

Diamond symbol represents a true/false condition

Single alternative decision structure provides only one alternative path of execution.

Figure A simple decision structure

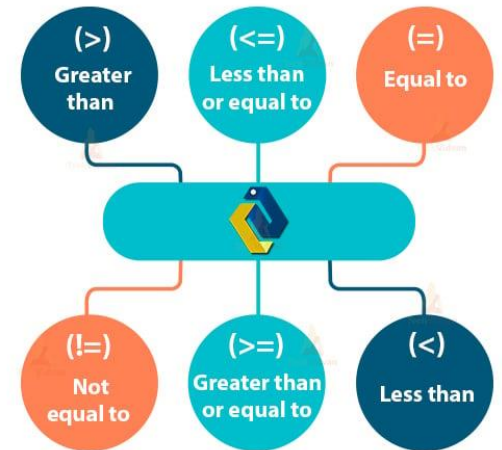


The Relational Operators

A ***relational operator*** determines whether a specific relationship exists between two values.

Operator	Meaning
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to
==	Equal to
!=	Not equal to

PYTHON RELATIONAL OPERATORS



The Relational Operators

Boolean Expressions and Relational Operators

Table 4-2 Boolean expressions using relational operators

Expression	Meaning
$x > y$	Is x greater than y ?
$x < y$	Is x less than y ?
$x \geq y$	Is x greater than or equal to y ?
$x \leq y$	Is x less than or equal to y ?
$x == y$	Is x equal to y ?
$x != y$	Is x not equal to y ?

The `if` Statement

Concept:

The `if` statement is used to create a decision structure, which allows a program to have more than one path of execution. The `if` statement causes one or more statements to execute only when a Boolean expression is true.

The `if` Statement

Decision (selection) structure:

In Python ...

*if condition:
statement
statement
statement
etc.*

```
def main():  
    # Get the three test scores.  
    test1 = input('Enter the score for test 1: ')  
    test2 = input('Enter the socre for test 2: ')  
    test3 = input('Enter the score for test 3: ')  
  
    # Calculate the average test score.  
    average = (test1 + test2 + test3) / 3.0  
  
    # Print the average.  
    print 'The average score is', average  
  
    # If the average is 95 or greater,  
    # congratulate the user.  
    if average >= 95:  
        print 'Congratulations!'  
        print 'That is a great average!'  
  
# Call the main function.  
main()
```

← This is the main
function's block.

← This is the `if`
statement's block.

The `if-else` Statement

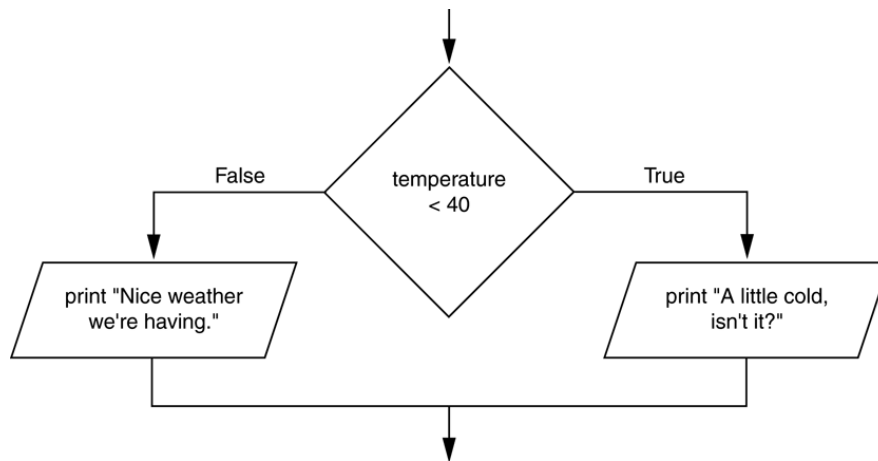
Concept:

An if-else statement will execute one block of statements if its condition is true, or another block if its condition is false.

The if-else Statement

A ***dual alternative*** decision structure provides two possible paths of execution – one path is taken if a condition is true, and the other path is taken if the condition is false.

Figure 4-6 A dual alternative decision structure

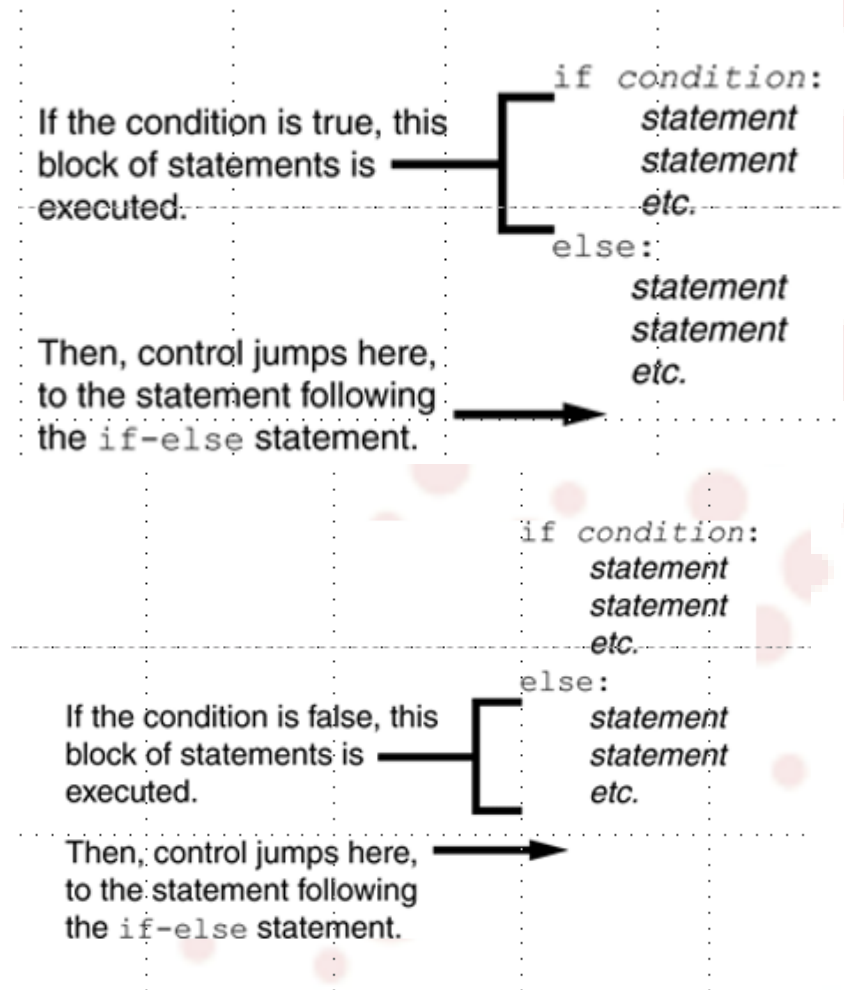


The if-else Statement

Dual alternative

In Python ...

```
if condition:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```



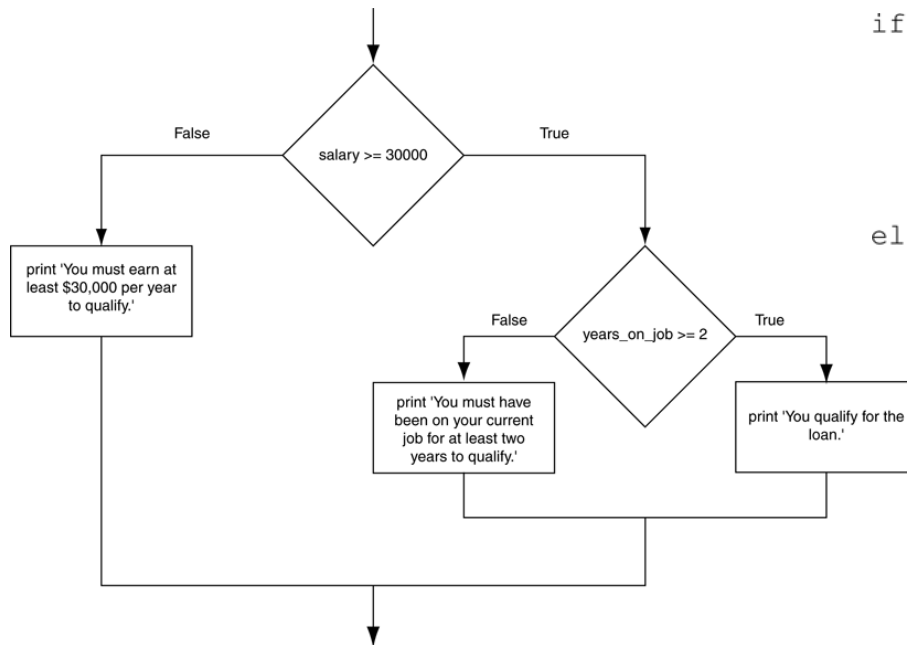
Nested Decision Structures and the `if-elif-else` Statement

Concept:

To test more than one condition, a decision structure can be nested inside another decision structure.

Nested Decision Structures and the `if-elif-else` Statement

Figure 4-14 A nested decision structure



```
if salary >= 30000.0:
    if years_on_job >= 2:
        print 'You qualify for the loan.'
    else:
        print 'You must have been on your current'
        print 'job for at least two years to qualify.'
else:
    print 'You must earn at least $30,000 per year'
    print 'to qualify.'
```

Nested Decision Structures and the if-elif-else Statement

The if-elif-else Statement

```
if score < 60:
    print 'Your grade is F.'
elif score < 70:
    print 'Your grade is D.'
elif score < 80:
    print 'Your grade is C.'
elif score < 90:
    print 'Your grade is B.'
else:
    print 'Your grade is A.'
```

```
if condition_1:
    statement
    statement
    etc.
elif condition_2:
    statement
    statement
    etc.
else:
    statement
    statement
    etc.
```

Logical Operators

Concept:

The logical **and** operator and the logical **or** operator allow you to connect multiple Boolean expressions to create a compound expression. The logical **not** operator reverses the truth of a Boolean expression.

Logical Operators

Table 4-3 Logical operators

Operator	Meaning
and	The and operator connects two Boolean expressions into one compound expression. Both subexpressions must be true for the compound expression to be true.
or	The or operator connects two Boolean expressions into one compound expression. One or both subexpressions must be true for the compound expression to be true. It is only necessary for one of the subexpressions to be true, and it does not matter which.
not	The not operator is a unary operator, meaning it works with only one operand. The operand must be a Boolean expression. The not operator reverses the truth of its operand. If it is applied to an expression that is true, the operator returns false. If it is applied to an expression that is false, the operator returns true.

Logical Operators

Table 4-4 Compound Boolean expressions using logical operators

Expression	Meaning
<code>x > y and a < b</code>	Is <code>x</code> greater than <code>y</code> AND is <code>a</code> less than <code>b</code> ?
<code>x == y or x == z</code>	Is <code>x</code> equal to <code>y</code> OR is <code>x</code> equal to <code>z</code> ?
<code>not (x > y)</code>	Is the expression <code>x > y</code> NOT true?

Logical Operators

The and Operator

Table 4-5 Truth table for the and operator

Expression	Value of the Expression
true and false	false
false and true	false
false and false	false
true and true	true

```
if temperature < 20 and minutes > 12:  
    print 'The temperature is in the danger zone.'
```

Logical Operators

The `or` Operator

Table 4-6 Truth table for the `or` operator

Expression	Value of the Expression
true or false	true
false or true	true
false or false	false
true or true	true

```
if temperature < 20 or temperature > 100:  
    print 'The temperature is too extreme.'
```

Logical Operators

The not Operator

Table 4-7 Truth table for the `not` operator

Expression	Value of the Expression
<code>not true</code>	false
<code>not false</code>	true

```
if not(temperature > 100):  
    print 'This is below the maximum temperature.'
```

Logical Operators

Checking Numeric Ranges with Logical Operators

```
if x >= 20 and x <= 40:  
    print 'The value is in the acceptable range.'  
  
if x < 20 or x > 40:  
    print 'The value is outside the acceptable range.'  
  
# This is an error!  
if x < 20 and x > 40:  
    print 'The value is outside the acceptable range.'
```

Boolean Variables

Concept:

A Boolean variable can reference one of two values: **True** or **False**. Boolean variables are commonly used as flags, which indicate whether a specific condition exists.

Boolean Variables

```
if sales >= 50000:  
    sales_quota_met = True  
else:  
    sales_quota_met = False
```

```
if sales_quota_met == True:  
    print 'You have met your sales quota!.'
```

OR

```
if sales_quota_met:  
    print 'You have met your sales quota!.'
```


2. Match Case in Python version 3.0

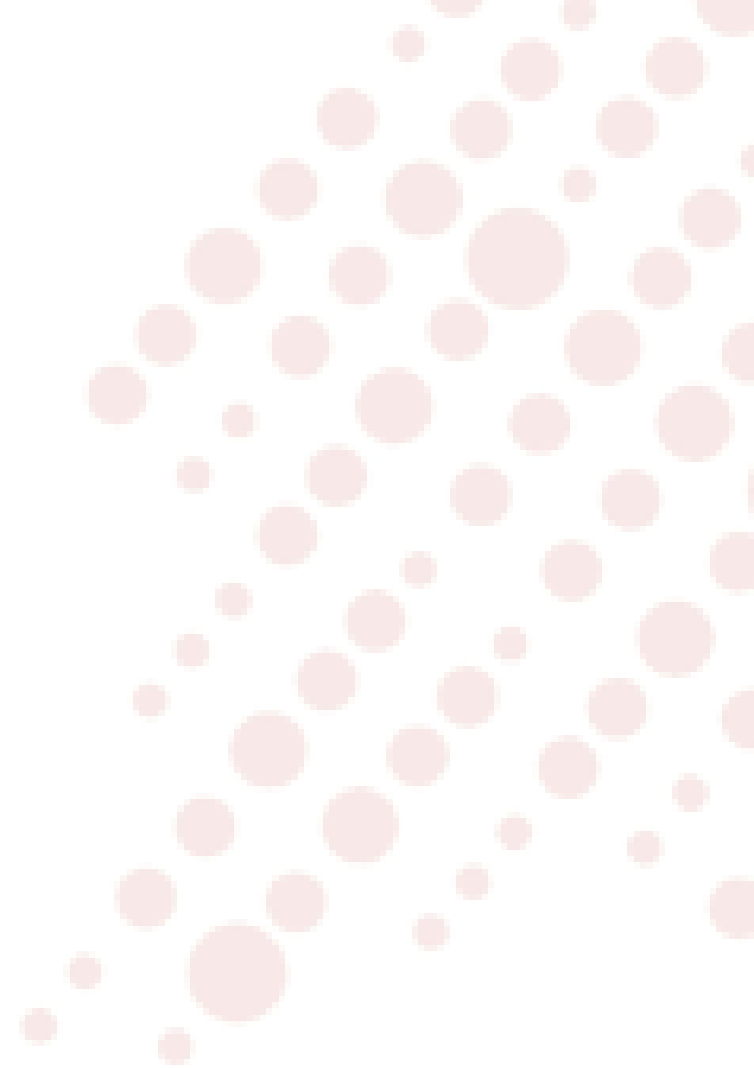
Pattern matching(match case in python)

```
match subject:
    case <pattern_1>:
        <action_1>
    case <pattern_2>:
        <action_2>
    case <pattern_3>:
        <action_3>
    case _:
        <action_wildcard>
```

```
PS G:\00 Swinburne\COS10009\00 Week Slides> python3 match.py
two
```

```
1 def main():
2     value = 2
3
4     match value:
5         case 1:
6             result = "one"
7         case 2:
8             result = "two"
9         case 3:
10            result = "three"
11        case _:
12            result = "unknown"
13
14    print(result) # Output: two
15 # end main
16
17 main()
```

Any Question?



Repetition Structures

3. Introduction to Repetition Structures

Concept:

The repetition structure causes a statement or set of statements to execute repeatedly.

Introduction to Repetition Structures

Condition-Controlled and Count-Controlled Loops

A *condition-controlled loop* uses a true/false condition to control the number of times that it repeats.

A *count-controlled loop* repeats a specific number of times.

The `while` Loop: a Condition-Controlled Loop

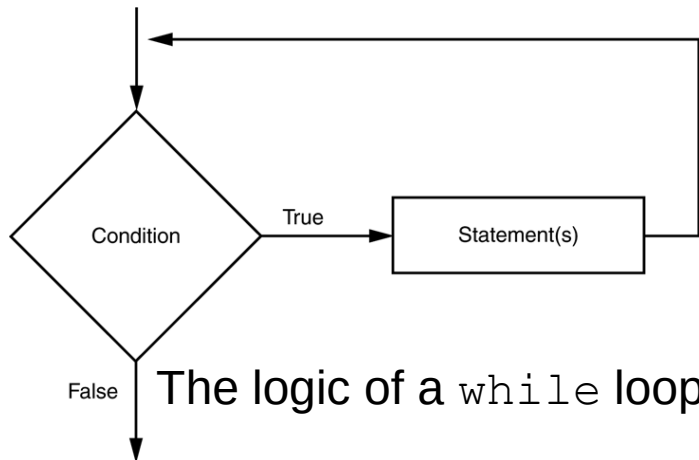
Concept:

A condition-controlled loop causes a statement or set of statements to repeat as long as a condition is true. In Python you use the **`while`** statement to write a condition-controlled loop.

The while Loop: a Condition-Controlled Loop

In Python ...

*while condition:
statement
statement
etc.*



```
number = 0
while number < 5:
    print ('I love to code')
    number += 1
```

Reset

Show answer

Run

Run executed.

```
I love to code
I love to code
I love to code
I love to code
I love to code
```

The while Loop: a Condition-Controlled Loop

The while
Loop is a
Pretest Loop,
which means it
tests its
condition before
performing an
iteration.

If the condition is true,
these statements are
executed, and then the
loop starts over.

If the condition is false,
these statements are
skipped and the
program exits the loop.

This condition is tested.

```
while keep_going == 'y':  
    # Get a salesperson's sales and commission rate.  
    sales = input('Enter the amount of sales: ')  
    comm_rate = input('Enter the commission rate: ')  
  
    # Calculate the commission.  
    commission = sales * comm_rate  
  
    # Display the commission  
    print 'The commission is $%.2f.' % commission  
  
    # See if the user wants to do another one.  
    keep_going = raw_input('Do you want to calculate another ' +  
                           'commission (Enter y for yes): ')
```


The `for` Loop: a Count-Controlled Loop

Concept:

A count-controlled loop iterates a specific number of times. In Python you use the `for` statement to write a count-controlled loop.

The for Loop: a Count-Controlled Loop

In Python ...

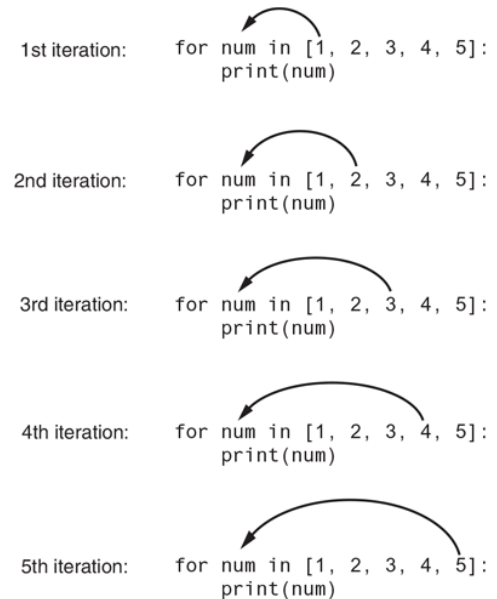
```
for variable in [value1, value2, etc.]:  
    statement  
    statement  
    etc.
```

The for loop

```
1 # This program also demonstrates a simple for  
2 # loop that uses a list of numbers.  
3  
4 print('I will display the odd numbers 1 through 9.')  
5 for num in [1, 3, 5, 7, 9]:  
6     print(num)
```

Output

```
I will display the odd  
numbers 1 through 9.  
1  
3  
5  
7
```



The `for` Loop: a Count-Controlled Loop

Using the `range` Function with the `for` Loop

```
for num in [0, 1, 2, 3, 4]:  
    print num
```

```
for num in range(5):  
    print num
```

The for Loop: a Count-Controlled Loop

Using the range Function with the for Loop

```
for num in range(1, 10, 2):  
    print num
```

- First argument, 1, is the starting value for the list
- Second argument, 10, is the ending limit of the list
- Third argument, 2, is the step value

```
1  # This program uses a loop to display a  
2  # table of numbers and their squares.  
3  
4  def main():  
5      # Get the starting value.  
6      print 'This program displays a list of numbers'  
7      print 'and their squares.'  
8      start = input('Enter the starting number: ')  
9  
10     # Get the ending limit.  
11     end = input('How high should I go? ')  
12  
13     # Print the table headings.  
14     print  
15     print 'Number\tSquare'  
16     print '-----'  
17  
18     # Print the numbers and their squares.  
19     for number in range(start, end + 1):  
20         square = number**2  
21         print number, '\t', square  
22  
23     # Call the main function.  
24     main()
```

The `for` Loop: a Count-Controlled Loop

Generating Lists that Range from Highest to Lowest

```
for num in range(5, 0, -1):  
    print num
```

The Augmented Assignment Operators

The Augmented Assignment Operators

Table 5-2 The age variable references the value 25

Operator	Example Usage	Equivalent To
<code>+=</code>	<code>x += 5</code>	<code>x = x + 5</code>
<code>-=</code>	<code>y -= 2</code>	<code>y = y - 2</code>
<code>*=</code>	<code>z *= 10</code>	<code>z = z * 10</code>
<code>/=</code>	<code>a /= b</code>	<code>a = a / b</code>
<code>%=</code>	<code>c %= 3</code>	<code>c = c % 3</code>

Input Validation Loops

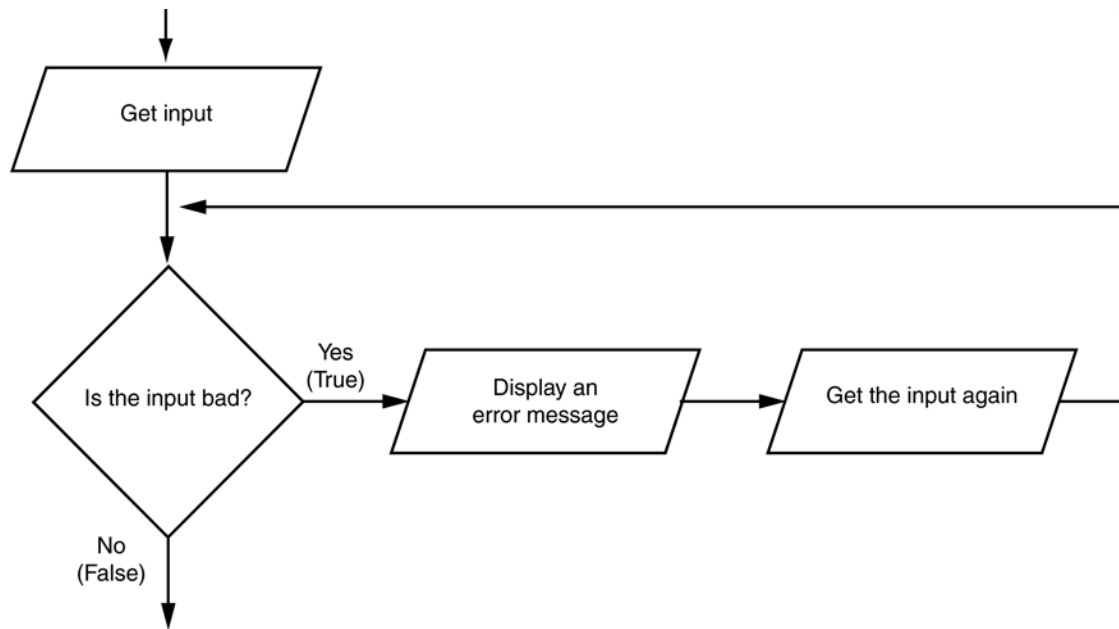
Concept:

Input validation is the process of inspecting data that has been input to a program, to make sure it is valid before it is used in a computation. Input validation is commonly done with a loop that iterates as long as an input variable references bad data.

Input Validation Loops

Error Trap or Error Handler

Figure 5-8 Logic containing an input validation loop



Input Validation Loops

Priming Read

```
# Get a test score.  
Score = input('Enter a test score: ')  
  
# Make sure it is not less than 0.  
while score < 0:  
    print 'ERROR: The score cannot be negative.'  
    score = input ('Enter the correct score: ')
```

Nested Loops

Concept:

A loop that is inside another loop is called a nested loop.

Nested Loops

```
for hours in range(24):  
    for minutes in range(60):  
        for seconds in range(60):  
            print hours, ':', minutes, ':',  
seconds
```

About Nested Loops:

- The inner loop goes through all of its iterations for every single iteration of the outer loop
- Inner loops complete their iterations faster than outer loops

Repetition Structures

QUESTIONS

?

4. Processing Lists with control structure

Processing Lists: Totaling the Values in a List

Program 8-16 (total_list.py)

```
1  # This program calculates the total of the values
2  # in a list.
3
4  def main():
5      # Create a list.
6      numbers = [2, 4, 6, 8, 10]
7
8      # Create a variable to use as an accumulator.
9      total = 0
10
11     # Calculate the total of the list elements.
12     for value in numbers:
13         total += value
14
15     # Display the total of the list elements.
16     print 'The total of the elements is', total
17
18 # Call the main function.
19 main()
```

Processing Lists with control structure

Averaging the Values in a List

Program 8-17 (average_list.py)

```
1  # This program calculates the average of the values
2  # in a list.
3
4  def main():
5      # Create a list.
6      scores = [2.5, 8.3, 6.5, 4.0, 5.2]
7
8      # Create a variable to use as an accumulator.
9      total = 0.0
10
11     # Calculate the total of the list elements.
12     for value in scores:
13         total += value
14
15     # Calculate the average of the elements.
16     average = total / len(scores)
17
18     # Display the total of the list elements.
19     print 'The average of the elements is', average
20
21 # Call the main function.
22 main()
```

Processing Lists with control structure

Search a Value in a List

CONCEPT: You can search for an item in a list using the `in` operator.

In Python, you can use the `in` operator to determine whether an item is contained in a list. Here is the general format of an expression written with the `in` operator to search for an item in a list:

```
item in list
```

In the general format, *item* is the item for which you are searching, and *list* is a list.

```
2 # used with a list.
3
4 def main():
5     # Create a list of product numbers.
6     prod_nums = ['V475', 'F987', 'Q143', 'R688']
7
8     # Get a product number to search for.
9     search = input('Enter a product number: ')
10
11     # Determine whether the product number is in the list.
12     if search in prod_nums:
13         print(f'{search} was found in the list.')
14     else:
15         print(f'{search} was not found in the list.')
16
17 # Call the main function.
18 if __name__ == '__main__':
19     main()
```

Program Output (with input shown in bold)

```
Enter a product number: Q143 [Enter]
Q143 was found in the list.
```

Program Output (with input shown in bold)

```
Enter a product number: B000 [Enter]
B000 was not found in the list.
```

Menu -1

How the Menu System Works

- The program uses a **menu-driven approach** to control execution.
 - A **Main Menu** is displayed inside a while loop and repeats until the user selects **Exit**.
 - User input is validated using `read_integer_in_range`.
 - The program uses `match / case` to decide which function to run.
- A **Sub Menu** (List Processing Menu) works the same way:
 - It has its own loop.
 - Each option calls a separate function.
 - Choosing “Return to Main Menu” exits the sub menu.
 - Input functions are stored in a **separate file**, keeping the menu logic clean and easy to read.

```
=====
# Main Menu
# =====
def main():
    finished = False

    while finished == False:
        print("\nMain Menu:")
        print("1 List Processing")
        print("2 Other Feature")
        print("3 Exit")

        choice =
input_functions.read_integer_in_range("Please enter
your choice:", 1, 3)

        match choice:
            case 1:
                list_processing_menu()
            case 2:
                other_feature()
            case 3:
                finished = True
            case _:
                print("Please select again")

main()
```


Menu -2

```
import input_functions

# =====
# List Processing Sub Menu
# =====
def list_processing_menu():
    finished = False

    while finished == False:
        print("\nList Processing Menu:")
        print("1 Enter a list")
        print("2 Calculate total")
        print("3 Calculate average")
        print("4 Search in list")
        print("5 Return to Main Menu")

        choice =
input_functions.read_integer_in_range("Please enter your
choice:", 1, 5)

        match choice:
            case 1:
                enter_list()
            case 2:
                calculate_total()
            case 3:
                calculate_average()
            case 4:
                search_item()
            case 5:
                finished = True
            case _:
                print("Please select again")
```

```
# =====
# Stub functions (EXPLANATION ONLY)
# =====
def enter_list():
    print("This is the code for ENTERING a list.")
    input("Press Enter to continue")

def calculate_total():
    print("This is the code for CALCULATING the total of a
list.")
    input("Press Enter to continue")

def calculate_average():
    print("This is the code for CALCULATING the average of a
list.")
    input("Press Enter to continue")

def search_item():
    print("This is the code for SEARCHING an item in a
list.")
    input("Press Enter to continue")

# =====
# Option 2 - Other Feature (Stub)
# =====
def other_feature():
    print("This is the code for OPTION 2 in the Main Menu.")
    input("Press Enter to continue")
```

Process List(full code) with Menu - 1

```
# =====  
# Main Menu (Ruby-style)  
# =====  
def main():  
    finished = False  
  
    while finished == False:  
        print("\nMain Menu:")  
        print("1 List Processing")  
        print("2 Other Feature")  
        print("3 Exit")  
  
        choice = input_functions.read_integer_in_range("Please enter  
your choice:", 1, 3)  
  
        match choice:  
            case 1:  
                list_processing_menu()  
            case 2:  
                other_feature()  
            case 3:  
                finished = True  
            case _:  
                print("Please select again")  
  
main()
```

Process List(full code) with Menu - 2

```
import input_functions

# =====
# Input list function
# =====
def input_number_list():
    numbers = []
    count = input_functions.read_integer_in_range("How many
numbers do you want to enter?", 1, 100)

    i = 0
    while i < count:
        value = float(input(f"Enter number {i + 1}: "))
        numbers.append(value)
        i += 1

    return numbers

# =====
# Processing functions
# (return only, no print)
# =====
def total_list(numbers):
    total = 0
    i = 0
    while i < len(numbers):
        total += numbers[i]
        i += 1
    return total

def average_list(numbers):
    return total_list(numbers) / len(numbers)
```

```
def search_list(numbers, target):
    i = 0
    while i < len(numbers):
        if numbers[i] == target:
            return True
        i += 1
    return False
```

Process List(full code) with Menu -3

```
# =====
# List Processing Sub Menu
# =====
def list_processing_menu():
    finished = False
    numbers = []

    while finished == False:
        print("\nList Processing Menu:")
        print("1 Enter a list of numbers")
        print("2 Calculate total of the list")
        print("3 Calculate average of the list")
        print("4 Search for a value in the list")
        print("5 Return to Main Menu")

        choice = input_functions.read_integer_in_range("Please
enter your choice:", 1, 5)

        match choice:
            case 1:
                numbers = input_number_list()

            case 2:
                if len(numbers) == 0:
                    print("The list is empty. Please enter a
list first.")
                else:
                    print("The total of the elements is",
total_list(numbers))
                    input("Press Enter to continue")

            case 3:
                if len(numbers) == 0:
                    print("The list is empty. Please enter a
list first.")
                else:
                    print("The average of the elements is",
average_list(numbers))
                    input("Press Enter to continue")
```

```
            case 4:
                if len(numbers) == 0:
                    print("The list is empty. Please enter a list
first.")
                else:
                    value = float(input("Enter a value to search for:
"))
                    if search_list(numbers, value):
                        print(value, "was found in the list.")
                    else:
                        print(value, "was not found in the list.")
                    input("Press Enter to continue")

            case 5:
                finished = True

            case _:
                print("Please select again")

# =====
# Option 2 - Other Feature (Stub)
# =====
def other_feature():
    print("You selected Option 2 (Other Feature).")
    input("Press Enter to return to the Main Menu")
```

QUESTIONS

?

Recap

- What is in this lesson?
1. The Decision Structure
 - **The Relational Operators**
 - **The if Statement**
 - **The if-else Statement**
 - **Logical Operators**
 - **The Match Case in Python version 3.0**
 2. Introduction to Repetition Structures
 - **The while Loop**
 - **The for Loop:**
 3. Processing List with Menu

